



SECJ2013

DATA STRUCTURE AND ALGORITHM

SECTION 02

ASSIGNMENT 1

LECTURER:

DR. ZURAINI BINTI ALI SHAH

GROUP MEMBER:

LIM YU HAN (A23CS0241)

EVELYN GOH YUAN QI (A23CS0222)

ANIS SAFIYYA BINTI JANAI (A23CS0049)

Table of Contents

1.Sorting Algorithms	3
1.1Bubble Sort	3
1.2 Improved Bubble Sort	4
2.Analysis Report	8
2.1Performance Comparison	8
2.2Efficiency of each algorithm.....	10
2.3 Summarizing Table	12

1. Sorting Algorithms

1.1 Bubble Sort

```
1 // bubble sort
2
3 #include <iostream>
4 using namespace std;
5
6 void BubbleSort (int data[], int listSize)
7 {
8     int pass, tempValue;
9     int comparisonCount=0;
10    int swapCount=0;
11
12    for (pass = 1; pass < listSize; pass++)
13    { // moves the largest element to the end of the array
14        for (int x = 0; x < listSize - pass; x++)
15        { // compare adjacent elements
16            comparisonCount++;
17            if ( data[x] > data[x+1] )
18            { // swap elements
19                tempValue = data[x];
20                data[x] = data[x+1];
21                data[x+1] = tempValue;
22
23                swapCount++;
24            }
25        }
26    }
27
28    cout << "\nTotal number of comparisons: " << comparisonCount << endl;
29    cout << "Total number of swaps: " << swapCount << endl;
30
31 } // end Bubble Sort
32
33
34 void printArray(int data[], int listSize)
35 {
36     for(int i = 0; i < listSize; i++)
37     {
38         cout << data[i] << " ";
39     }
40     cout << endl;
41 }
42
43 int main()
44 {
45     int studentMarks[] = {75,95,60,88,70};
46     int list=5;
47
48     cout << "Original array: ";
49     printArray(studentMarks, list);
50
51     BubbleSort(studentMarks, list);
52
53     cout << "\nBubble Sorted array in ascending order: ";
54     printArray(studentMarks, list);
55
56     return 0;
57 }
```

Output:

Original array: 75 95 60 88 70

Total number of comparisons: 10

Total number of swaps: 6

Bubble Sorted array in ascending order: 60 70 75 88 95

1.2 Improved Bubble Sort

```
#include <iostream>
using namespace std;

void improvedBubbleSort (int data[], int n)
{
    int pass,temp;
    int comparisonCount=0;
    int swapCount=0;
    bool sorted = false; // false when swaps occur
    for (int pass = 1; (pass < n) && !sorted; ++pass)
    {
        sorted = true; // assume sorted
        for (int x = 0; x < n-pass; ++x)
        {
            comparisonCount++;
            if (data[x] > data[x+1])
            {
                // exchange items
                temp = data[x];
                data[x] = data[x+1];
                data[x+1] = temp;
                sorted = false; // signal exchange
                swapCount++;
            } // end if
        } // end for
    } //outer for
    cout<<"\nTotal number of comparisons: "<<comparisonCount<<endl;
    cout<<"Total number of swap: "<<swapCount<<endl;
} //end

void printArray(int data[], int listSize)
{
    for (int i=0; i<listSize; i++)
    {
        cout<<data[i]<<" ";
    }
    cout<<endl;
}

int main()
{
    int studentMarks[]={75,95,60,88,70}; //unsorted data

    int list = 5;
    cout<<"Original array: ";
    printArray(studentMarks,list);

    improvedBubbleSort(studentMarks,list);

    cout<<"\nImproved bubble sorted array in ascending order: ";
    printArray(studentMarks,list);

    return 0;
}
```

Output:

```
Original array: 75 95 60 88 70
Total number of comparisons: 10
Total number of swap: 6

Improved bubble sorted array in ascending order: 60 70 75 88 95

-----
Process exited after 0.7206 seconds with return value 0
Press any key to continue . . .
```

1.3 Selection Sort

```
#include <iostream>
using namespace std;

void selectionSort(int Data[], int n)
{
    int comparisonNum = 0;
    int swapNum = 0;

    for (int last = n-1; last >= 1; --last)
    {
        // select largest item in the Array
        int largestIndex = 0;

        // largest item is assumed start at index 0
        for (int p=1; p <= last; ++p)
        {
            if (Data[p] > Data[largestIndex])
                largestIndex = p;
            ++comparisonNum;
        }

        // swap largest item Data[largestIndex] with Data[last]
        if (Data[largestIndex] != Data[last])
        {
            swap(Data[largestIndex], Data[last]);
            ++swapNum;
        }
    }

    cout<< "\nTotal number of comparisons: " << comparisonNum;
    cout<< "\nTotal number of swaps: " << swapNum;
} // end selectionSort
```

```
void swap(int& x, int& y)
{
    int temp = x;
    x = y;
    y = temp;
} // end swap

void printArray(int Data[], int arraySize)
{
    for (int i = 0; i < arraySize; i++)
    {
        cout<< Data[i] << " ";
    }
    cout<< endl;
}

int main()
{
    int studentMarks[] = {75, 95, 60, 88, 70};
    int arraySize = 5;        // 5 student marks

    cout<<"Unsorted list of student marks: ";
    printArray(studentMarks, arraySize);

    selectionSort(studentMarks, arraySize);

    cout<<"\n\nSorted list of student marks: ";
    printArray(studentMarks, arraySize);

    system("pause");
    return 0;
}
```

Output:

Unsorted list of student marks: 75 95 60 88 70

Total number of comparisons: 10

Total number of swaps: 2

Sorted list of student marks: 60 70 75 88 95

Press any key to continue . . .

1.4 Insertion Sort

```
1  #include <iostream>
2  using namespace std;
3
4  void inser(int x[], int n, int &comp, int &swap)
5  {
6      comp = 0; swap = 0;
7      for (int i = 1; i < n; i++)
8      {
9          int y = x[i];
10         int j = i - 1;
11
12         while (j >= 0 && x[j] > y)
13         {
14             comp++;
15             x[j + 1] = x[j];
16             swap++;
17             j = j - 1;
18         }
19
20         comp++;
21         x[j + 1] = y;
22         if (j + 1 != i)
23         {
24             swap++;
25         }
26     }
27 }
28
29 void printArray(int x[], int n)
30 {
31     for (int i = 0; i < n; i++)
32     {
33         cout << x[i] << " ";
34     }
35     cout << endl;
36 }
37
38 int main()
39 {
40     int x[] = {75, 95, 60, 88, 70};
41     int n = sizeof(x) / sizeof(x[0]);
42
43     int comp = 0, swap = 0;
44
45     cout << "Unsorted array: ";
46     printArray(x, n); // Print the array before sorting
47
48     inser(x, n, comp, swap);
49
50     cout << "Sorted array: ";
51     printArray(x, n); // Print the array after sorting
52
53     cout << "Total comparisons: " << comp << endl;
54     cout << "Total swaps: " << swap << endl;
55
56     system("pause");
57
58     return 0;
59 }
```

Output :

```
Unsorted array: 75 95 60 88 70
Sorted array: 60 70 75 88 95
Total comparisons: 10
Total swaps: 9
Press any key to continue . . .
```

2. Analysis Report

2.1 Performance Comparison

Bubble Sort

- **Number of comparisons:** The algorithm performed a total of 10 comparisons for a list of 5. This matches the expected performance of Bubble Sort, where the number of comparisons for a list of size n is approximately $(n - 1) + (n - 2) + \dots + 2 + 1 = (n(n - 1)) / 2$.
- **Number of swaps:** The total number of swaps required was 6 for this dataset.

Bubble Sort systematically compares and swaps adjacent elements as needed, ensuring the largest unsorted element “bubbles” to the correct position with each pass.

Improved Bubble Sort

- **Number of Comparisons:** The algorithm performed a total of 10 comparisons for a list of 5 elements. This is identical to the standard Bubble Sort for this dataset, as the array required multiple passes to become fully sorted, and the early termination condition did not come into effect until the final pass.
- **Number of Swaps:** The total number of swaps required was 6, matching the standard Bubble Sort for this dataset. Since every comparison that identified a mismatch led to a swap, the swap count remained unchanged.

Improved Bubble Sort incorporates an optimization by using a flag (sorted) to detect whether any swaps occurred in the current pass. If no swaps are made, the algorithm terminates early, as this indicates the array is already sorted. While this optimization significantly reduces redundant passes for partially sorted or already sorted datasets, it did not show any performance advantage in this test case due to the dataset being completely unsorted at the start.

Selection Sort

- **Number of comparisons:** The algorithm performed a total of 10 comparisons for a list of 5 elements. This fits the expected performance of Selection Sort, where the number of comparisons for a list of size n is approximately $(n - 1) + (n - 2) + \dots + 2 + 1 = (n(n - 1)) / 2$.
- **Number of swaps:** The total number of swaps required was 2 for this dataset.

Selection Sort compares all the elements in the list to find the largest value and swaps the element to the last position of the subarray repetitively in order to sort all the elements.

Insertion Sort

- **Number of comparisons:** The algorithm performed a total of 10 comparisons for a list of 5 elements. This aligns with the expected performance of Insertion Sort, where the number of comparisons depends on the initial order of the elements, typically ranging between $n-1$ (best case) and $n(n-1)/2$ (worst case).
- **Number of swaps:** The total number of swaps required was 9 for this dataset.

Insertion Sort builds the sorted array one element at a time, by comparing and shifting elements as needed to insert the current element into its correct position.

2.2 Efficiency of each algorithm

Sorting Algorithm	Efficiency of the algorithm
Bubble Sort	Requires $O(n^2)$ comparisons and $O(n^2)$ swaps, making it poor efficiency and not well-suited for larger datasets but remains effective for small or nearly sorted arrays.
Improve Bubble Sort	Requires $O(n^2)$ comparisons and $O(n^2)$ swaps in the worst case, similar to Bubble Sort, which limits its efficiency for larger datasets. However, its key advantage lies in the best-case scenario, where it requires only $O(n)$ comparisons and $O(1)$ swaps if the array is already sorted. This makes it better suited for small or partially sorted arrays compared to standard Bubble Sort.
Selection Sort	Requires $O(n^2)$ comparisons in all cases including the worst case, best case and average case as it does not depend on the initial arrangement of the data. Thus, partially sorted array will not affect the number of comparisons for selection sort. Furthermore, it performs the fewest swaps, $O(n)$, among the other sorts which makes it efficient for reducing write operations.
Insertion Sort	Insertion Sort requires $O(n^2)$ comparisons and $O(n^2)$ shifts in the worst case, similar to Bubble Sort, which limits its efficiency for larger datasets. However, its key advantage lies in the best-case scenario, where it requires only $O(n)$ comparisons and $O(1)$ shifts if the array is already sorted. This makes it better suited for small or partially sorted arrays compared to standard Bubble Sort.

2.3 Summarizing Table

Sorting Algorithm	Comparisons	Swaps
Bubble Sort	10	6
Improved Bubble Sort	10	6
Selection Sort	10	2
Insertion Sort	10	9